计算机视觉基础：方向梯度直方图 (HOG)

特征提取总结报告

2026 年 3 月 19 日

1 HOG (Histogram of Oriented Gradients) 概述

HOG 是一种在计算机视觉和图像处理中广泛使用的**特征描述符** (Feature Descriptor)，最初由 Dalal 和 Triggs 在 2005 年提出，主要用于行人检测任务。

其核心思想是：**物体的外形和外观可以通过强度梯度 (Gradients) 或边缘方向的分布来很好地描述**。即便图像在光照变化或背景复杂的情况下，只要能够捕捉到清晰的物体轮廓梯度信息，就能进行有效识别。

2 HOG 特征提取核心步骤

HOG 特征的计算主要包含以下几个关键阶段：

1. **预处理 (Preprocessing)：**通常将图像转换为灰度图，有时会进行 Gamma 校正或归一化，以减少局部光照和阴影的影响。

2. **计算梯度 (Compute Gradients)：**计算图像中每个像素点在水平方向 (G_x) 和垂直方向 (G_y) 的梯度。进而计算梯度的幅值和方向：

$$\text{幅值}M(x,y) = \sqrt{G_x^2 + G_y^2} \tag{1}$$

$$\text{方向}\theta(x,y) = \arctan\left(\frac{G_y}{G_x}\right) \tag{2}$$

3. **构建细胞单元直方图 (Create Cell Histograms)：**将图像划分为小的、不重叠的区域，称为**细胞单元 (Cells)** (例如 8×8 像素)。在每个 Cell 内，根据像素点的梯度方向进行投票，构建一个直方图 (通常将 $0^\circ - 180^\circ$ 分为 9 个 Bin)。

4. **块归一化 (Block Normalization)：**为了提高对光照变化的鲁棒性，将多个相邻的 Cell 组合成一个较大的、重叠的区域，称为**块 (Block)** (例如 2×2 个 Cells)。对 Block 内的所有特征向量进行归一化处理。

5. **生成总特征向量：**将所有 Blocks 的归一化特征向量拼接起来，形成最终用于分类的 HOG 特征向量。

3 Python 代码实现

以下是使用 scikit-image 库提取 HOG 特征并进行可视化的完整 Python 实现方案：

```python
import matplotlib.pyplot as plt
from skimage.feature import hog
from skimage import data, exposure
import cv2

# 1. 加载图像（以 skimage 自带的人脸图像为例）
image = data.astronaut()
image_gray = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY)

# 2. 计算 HOG 特征
# orientations: 直方图的槽数（Bin）0-180度，每20度为一个bin
fd, hog_image = hog(image_gray,
                    orientations=9,
                    pixels_per_cell=(8, 8),
                    cells_per_block=(2, 2),
                    visualize=True,
                    channel_axis=None)

# 3. 结果可视化
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 6), sharex=True, sharey=True)
ax1.axis('off')
ax1.imshow(image_gray, cmap=plt.cm.gray)
ax1.set_title('Original Image')

# 对 HOG 图像进行强度缩放，方便观察
hog_image_rescaled = exposure.rescale_intensity(hog_image, in_range=(0, 10))
ax2.axis('off')
ax2.imshow(hog_image_rescaled, cmap=plt.cm.gray)
ax2.set_title('HOG Visualization')
plt.show()
print(f"特征向量的维度是: {fd.shape}")
```

Listing 1: HOG 特征提取与可视化脚本

4 代码分析说明

本节对所提供的 Python 代码进行详细解析，探讨其在特征提取过程中的具体实现逻辑与参数配置。

4.1 1. 图像预处理 (Preprocessing)

代码首先通过 data.astronaut() 获取一张 512×512 像素的彩色图像。

- **灰度化处理：** 使用 cv2.cvtColor 将图像转为灰度图。由于 HOG 关注的是物体的形状和边缘（即梯度的变化），忽略颜色信息可以显著减少计算开销，同时提高对不同光照条件的鲁棒性。

4.2 2. HOG 核心参数配置

调用 `skimage.feature.hog` 函数时，设置了三个决定特征质量的关键参数：

- `orientations=9`：将 180° 的梯度方向划分为 9 个 Bin（每 20° 一个），用于捕捉局部边缘的方向性。
- `pixels_per_cell=(8, 8)`：设定最小统计单位（Cell）为 8×8 像素，这决定了特征提取的粒度。
- `cells_per_block=(2, 2)`：设定归一化单位（Block）由 2×2 个 Cells 组成，通过重叠滑动（Overlap）来抵消光照强度变化的影响。

4.3 3. 特征维度的数学验证

实验运行结果

特征向量维度为：**142884**

其推导过程如下：

- **Cell 计数：** 在 512×512 图像中，Cells 数量为 $\frac{512}{8} \times \frac{512}{8} = 64 \times 64$。
- **Block 计数：** Block 以 1 个 Cell 为步长滑动，横向与纵向的 Block 数均为 $64 - 2 + 1 = 63$。总数为 $63 \times 63 = 3969$。
- **单 Block 维度：** 每个 Block 包含 $2 \times 2 = 4$ 个 Cells，每个 Cell 有 9 维向量，故单 Block 维度为 $4 \times 9 = 36$。
- **总维度：** $3969(\text{Blocks}) \times 36(\text{Dim/Block}) = 142884$。

4.4 4. 可视化处理 (Visualization)

代码中使用了 `exposure.rescale_intensity` 函数：

- **强度缩放：** 原始生成的 HOG 图像像素值通常极小。通过线性拉伸（将范围映射到 $0 - 10$），可以使人类视觉观察到由梯度方向构成的“轮廓图”。

5 实验结果展示

6 HOG 算法优缺点分析

6.1 主要优点 (Advantages)

- **几何与光学不变性：** 由于 HOG 是在图像的局部细胞单元（Cells）上操作，并经过了块归一化，它对图像的微小位移、旋转以及光照变化具有较强的鲁棒性。
- **强调边缘与轮廓：** HOG 通过捕捉梯度的方向分布，能够精准地描述物体的外形特征。
- **无需大规模训练数据：** 在数据集较小时，配合线性 SVM 分类器往往能取得比复杂神经网络更稳定的效果。

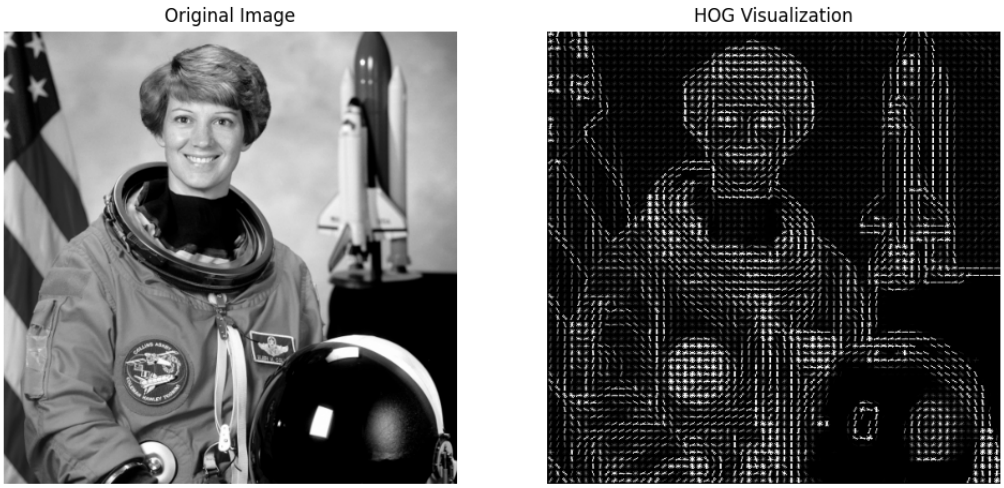


图 1: HOG 特征向量可视化结果

6.2 主要缺点 (Disadvantages)

- **计算开销较大：** 在提取特征时，需要遍历整张图像计算梯度、直方图。
- **旋转敏感性：** 虽然 HOG 对小范围位移鲁棒，但它并不是旋转不变的。
- **特征维度冗余：** 一张 512×512 的图片会生成高达 $142,884$ 维的特征，增加了计算负担。

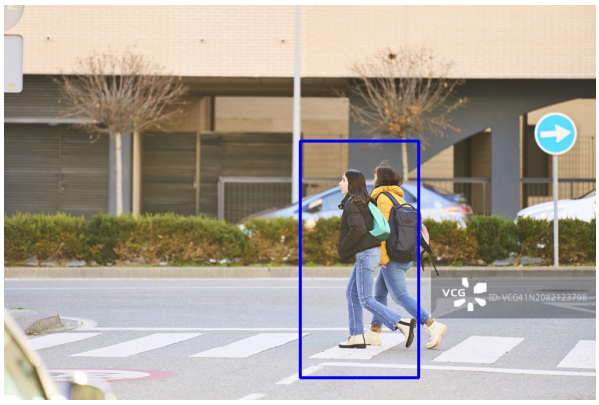
7 行人检测 (Pedestrian Detection) 应用分析

本实验中的行人检测通过调用 OpenCV 内置的 `HOGDescriptor_getDefaultPeopleDetector()` 预训练模型实现，该模型基于 64×128 像素的标准行人样本构建。在具体检测过程中，程序利用 `detectMultiScale` 函数构建图像金字塔，通过不断按比例（由 `scale` 参数控制）缩放原始图像来捕捉因拍摄距离不同而呈现出不同尺寸的目标。检测器在每一层金字塔上以设定的步长（`winStride`）进行滑动窗口扫描，并利用 `padding` 处理图像边缘。由于 HOG 特征对目标的微小位移具有较好的容忍度，同一个行人往往会被多个重叠的扫描窗口同时识别，因此实验最后引入了非极大值抑制（NMS）算法。该算法通过计算检测框之间的交并比（IoU）并参考 SVM 输出的置信度权重，剔除冗余的重复检测框，从而实现对目标行人的精确标注与融合。

```
# (此处省略部分重复定义的检测函数，逻辑同前文所述)
hog = cv2.HOGDescriptor()
hog.setSVMDetector(cv2.HOGDescriptor_getDefaultPeopleDetector())
# 执行多尺度检测
(rects, weights) = hog.detectMultiScale(image_resized, winStride=(4, 4), padding=(8, 8),
    scale=1.05)
```

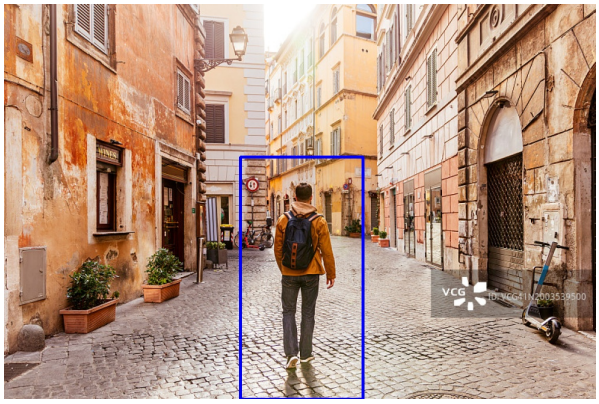
Listing 2: 行人检测完整实现代码

(a) 原始图像 1



(b) 检测结果 1

(c) 原始图像 2



(d) 检测结果 2

图 2: 基于 HOG+SVM 的行人检测实验结果对比图